

Plan de Despliegue y Mantenimiento

Entrega Final · Dispositivos Móviles · 2026

06-PLAN-DESPLIEGUE-MANTENIMIENTO

AUTORES

Andrés Botero

Tecnología en Desarrollo de Software

Juan Camilo Triana

Tecnología en Desarrollo de Software

PROYECTO

OutfitCatalog · ATELIER Fashion Catalog

ASIGNATURA

Dispositivos Móviles

FECHA

Junio 2026

VERSIÓN

1.0.0

Contenido

Plan de Despliegue y Mantenimiento

1. Objetivo
2. Entregables tecnicos
3. Requisitos de ambiente
4. Variables de entorno requeridas
5. Instalacion local
6. Generacion de APK
7. Publicacion en tiendas
8. Plan de soporte
9. Mantenimiento preventivo
10. Monitoreo recomendado
11. Plan de datos
12. Control de versiones
13. Riesgos post-entrega
14. Conclusion

1. Objetivo

Definir el procedimiento para desplegar, mantener y dar soporte a OutfitCatalog despues de la entrega final.

2. Entregables tecnicos

Entregable	Estado
Codigo fuente	Disponible en repositorio Git
APK Android	Generado
Documentacion tecnica	Incluida en docs/entrega-final
Pruebas automatizadas	Incluidas
Script de carga masiva	Incluido
Configuracion EAS	Incluida

3. Requisitos de ambiente

- ☐ Node.js LTS.
- ☐ npm.
- ☐ Cuenta Expo/EAS.
- ☐ Proyecto Firebase con Auth y Firestore.
- ☐ Variables de entorno configuradas.
- ☐ Cuenta Cloudinary si se usara subida de imagenes.
- ☐ Android Studio para pruebas locales en emulador.

4. Variables de entorno requeridas

TEXT

```
EXPO_PUBLIC_FIREBASE_API_KEY
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN
EXPO_PUBLIC_FIREBASE_PROJECT_ID
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
EXPO_PUBLIC_FIREBASE_APP_ID
EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID
EXPO_PUBLIC_CLOUDINARY_CLOUD_NAME
EXPO_PUBLIC_CLOUDINARY_UPLOAD
```

Servicios de try-on o IA pueden requerir:

TEXT

```
EXPO_PUBLIC_REPLICATE_TOKEN
EXPO_PUBLIC_SEGMIND_API_KEY
EXPO_PUBLIC_HF_TOKEN
```

5. Instalacion local

BASH

```
npm install
npm run build
npm test
npm run start
```

6. Generacion de APK

Comando validado:

BASH

```
npx eas-cli@latest build -p android --profile preview --non-interactive
```

Resultado final de esta entrega:

TEXT

```
https://expo.dev/artifacts/eas/hFhAPFShte8uvkEKgF2FTW.apk
```

7. Publicacion en tiendas

Google Play

Requisitos:

- ☐ Cuenta Google Play Console.
- ☐ App firmada.
- ☐ Ficha de tienda.
- ☐ Capturas.
- ☐ Politica de privacidad.
- ☐ Clasificacion de contenido.
- ☐ Pruebas internas o cerradas.

App Store

Requisitos:

- ☐ Cuenta Apple Developer.
- ☐ Build iOS con EAS.
- ☐ TestFlight.
- ☐ Ficha App Store Connect.
- ☐ Politica de privacidad.
- ☐ Capturas para iPhone.
- ☐ Revision de Apple.

8. Plan de soporte

Prioridad	Ejemplo	Tiempo sugerido
Critica	App no abre, login bloqueado, perdida de datos	24 horas
Alta	Solicitudes no se crean, inventario no actualiza	48 horas

Prioridad	Ejemplo	Tiempo sugerido
Media	Error visual, texto incorrecto, lentitud puntual	3 a 5 días
Baja	Mejora estetica o funcional menor	Siguiente version

9. Mantenimiento preventivo

- ❑ Revisar reglas de Firebase cada vez que se agregue una coleccion.
- ❑ Ejecutar npm run build antes de cada release.
- ❑ Ejecutar npm test antes de cada release.
- ❑ Probar APK en dispositivo real.
- ❑ Mantener dependencias Expo compatibles.
- ❑ Revisar cuotas de Firebase y Cloudinary.
- ❑ Hacer respaldo de Firestore antes de cargas masivas.

10. Monitoreo recomendado

La version final incluye analitica propia mediante Firestore en analyticsEvents. Para produccion se recomienda complementar esa base con:

- ❑ Firebase Crashlytics para cierres inesperados.
- ❑ Firebase Analytics nativo para eventos de usuario en tiendas.
- ❑ Performance Monitoring para tiempos de carga.
- ❑ Alertas de uso de Firestore.
- ❑ Revision periodica de errores de autenticacion.

11. Plan de datos

El script masivo permite preparar la base para pruebas:

```
BASH
npm run seed:massive -- --dry-run
```

Para escribir en Firebase con cuenta de servicio:

```
BASH
npm run seed:massive -- --service-account "ruta.json" --auth-users
```

Antes de ejecutar en produccion:

1. Confirmar proyecto Firebase correcto.
2. Hacer respaldo.
3. Ejecutar --dry-run.
4. Validar cantidades.
5. Ejecutar carga real.
6. Probar login y catalogo.

12. Control de versiones

Commit final:

TEXT

5bd8d25 feat: optimiza inventario y solicitudes de compra

Rama principal:

TEXT

main

Recomendacion:

- ☐ Usar ramas develop para cambios nuevos.
- ☐ Usar main solo para versiones estables.
- ☐ Etiquetar releases con v1.0.0, v1.0.1, etc.

13. Riesgos post-entrega

Riesgo	Mitigacion
Cambios en reglas Firebase bloquean flujos	Versionar reglas y probar matriz QA
API keys mal configuradas	Documentar variables y validar EAS environment
Publicacion iOS pendiente	Planificar build TestFlight
Crecimiento de catalogo	Mantener consultas filtradas y paginacion futura
Errores no visibles	Integrar Crashlytics

14. Conclusion

El despliegue Android esta resuelto mediante EAS Build y el APK final fue generado correctamente. Para una operacion productiva se recomienda completar monitoreo, analiticas, publicacion en tiendas y soporte formal.